CSM 61B Spring 2021

Final Review

Mentoring 15: April 26, 2021

1 Asymptotics

1.1 Give a tight asymptotic runtime bound for `mysterySearch` as a function of N, the size of the array, in the *best case, worst case, and overall*. Assume the array is sorted.

```
public static boolean mysterySearch(int[] a, int value) {
    if (Math.random() < 0.5) {
        return linearSearch(a, value, 0);
    } else {
        return binarySearch(a, value, 0, a.length);
    }
}
```

1.2 Give a tight asymptotic bound for `convoluted` as a function of N, the length of the input arrays `a` and `b`. If possible, give a $\Theta(\cdot)$ bound for the overall runtime. Otherwise, provide a $\Theta(\cdot)$ bound for both the best case and worst case runtime.

```
int[] convoluted(int[] a, int[] b) {
    assert a.length == b.length;
    int[] result = new int[a.length];
    for (int i = 0; i < result.length; i += 1) {
        for (int j = 0; j <= i; j += 1) {
            result[i] += a[j] * b[i];
        }
    }
    return result;
}
```

1.3 Give a tight asymptotic bound for `mystery`.

```
int mystery(int N) {
    if (N == 0) {
        return 0;
    }
    return mystery(N/3) + mystery(N/3) + mystery(N/3);
}
```

1.4 Give a tight asymptotic bound for `debugBinarySearch` as a function of N, the length of the input array, `a`. If possible, give a $\Theta(\cdot)$ bound for the overall runtime. Otherwise, provide a $\Theta(\cdot)$ bound for both the best case and worst case runtime.

```
boolean debugBinarySearch(int[] a, int target) {
    int start = 0;
    int end = a.length - 1;

    while (start <= end) {
        int mid = ((end - start) / 2) + start;
        if (a[mid] == target) {
            return true;
        } else if (a[mid] < target) {
            start = mid + 1;
        } else {
            end = mid - 1;
        }

        System.out.print("Searching: [ ");
        for (int i = start; i <= end; i++) {
            System.out.print(a[i] + " ");
        }
        System.out.println("]");
    }
    return false;
}
```

1.5 Give a tight asymptotic bound for `many_N` as a function of N and M. If possible, give a $\Theta(\cdot)$ bound for the overall runtime. Otherwise, provide a $\Theta(\cdot)$ bound for both the best case and worst case runtime.

```
int many_N(int N, int M) {
    if (N == 0) { return 1;}
    int k = 0;
    for (int i = 0; i <= M; i++) {
        k += many_N(N-1, M)
    }
    return k;
}
```

1.6 Give a tight asymptotic bound for `skip` as a function of N. If possible, give a $\Theta(\cdot)$ bound for the overall runtime. Otherwise, provide a $\Theta(\cdot)$ bound for both the best case and worst case runtime.

```
void skip(int N) {
    if (N <= 1) { return;}
    else {
        skip(N/N);
        skip(N/2);
    }
}
```

2 Sorting - Age Sort

2.1 For taxes, you must to submit a list of the ages of your customers in sorted order. Define `ageSort`, which takes an **`int[]`** `array` of all customers' ages and returns a sorted array. Assume customers are less than 150 years old.

```
public class BerkeleyBytes {
    public static int[] histogram(int[] ages) {
        private static int maxAge = 149;
        int[] ageCounts = new int[maxAge + 1];
        for (int age : ages) {
            ageCounts[age] += 1;
        }
        return ageCounts;
    }
    public static int[] ageSort(int[] ages) {









    }
}
```

3 What Sorts?

3.1 Each column below gives the contents of a list at some step during sorting. Match each column with its corresponding algorithm.

· Merge sort · Quicksort · Heap sort · LSD radix sort · MSD radix sort

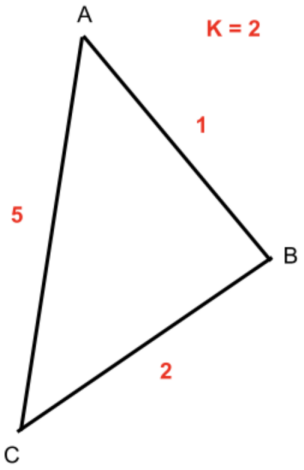
For quicksort, choose the topmost element as the pivot. Use the recursive (top-down) implementation of merge sort.

	Start	A	B	C	D	E	Sorted
1	4873	1876	1874	1626	9573	2212	1626
2	1874	1874	1626	1874	7121	8917	1874
3	8917	2212	1876	1876	9132	7121	1876
4	1626	1626	1897	4873	6973	1626	1897
5	4982	3492	2212	4982	4982	9132	2212
6	9132	1897	3492	8917	8917	6152	3492
7	9573	4873	4873	9132	6152	4873	4873
8	1876	9573	4982	9573	1876	9573	4982
9	6973	6973	6973	1897	1626	6973	6152
10	1897	9132	6152	3492	1897	1874	6973
11	9587	9587	7121	6973	1874	1876	7121
12	3492	4982	8917	9587	3492	9877	8917
13	9877	9877	9132	2212	4873	4982	9132
14	2212	8917	9573	6152	2212	9587	9573
15	6152	6152	9587	7121	9587	3492	9587
16	7121	7121	9877	9877	9877	1897	9877

4 Graphs

4.1 State if the following statements are True or False, and justify. For all graphs, assume that edge weights are positive and distinct, unless otherwise stated.

(a) Adding some positive constant k to every edge weight does not change the shortest path tree from vertex S.



(b) Doubling every edge weight does not change the shortest path tree.

(c) If the weight of each edge is decreased by 1, then the resulting shortest path in any graph from u to v is unchanged.

(d) If a graph G has a unique MST, it must have unique edge weights.

(e) Adding some positive constant k to every edge weight does not change the minimum spanning tree.

(f) Doubling every edge weight does not change the minimum spanning tree.

(g) Let $(S, V - S)$ be a specific cut of the graph. If an edge e is not the lightest edge across this cut, it cannot be a part of any MST.

(h) If an edge e is the lightest edge connected to vertex S, it must be a part of the shortest path tree from vertex S.

(i) Consider a graph G, where every edge is nonnegative, except the edges adjacent to vertex s. Dijkstra's usually fails on graphs with negative edge weights, however if we run Dijkstra's starting from s, we will get the correct shortest paths tree.

5 Weighted QuickUnion

5.1 Suppose we have a `WeightedQuickUnionUF` disjoint set with path compression. Show the tree structure in the union-find algorithm as the following sequence of commands is executed.

```
connect(1, 2);
connect(3, 4);
connect(5, 6);
connect(1, 6);
connect(3, 6);
```

5.2 Describe how to construct a `WeightedQuickUnionUF` tree of maximum height.